Design And Analysis of Algorithms
Lecture 6: Combinatorial Matching Algorithms.

February 14, 2018

In this lecture, we will study algorithms for constructing matchings on graphs. A matching $M \subseteq E$ has maximum degree at most 1. In the case of bipartite graphs, this problem can be reduced to solving max flow (and we can then use Ford-Fulkerson to solve it); we will briefly review this before considering König's algorithm, a combinatorial algorithm which will guide us in the design and analysis of Edmond's algorithm for computing maximum matchings on general graphs.

1 Current Best Algorithms

The current fastest algorithm for computing maximum matchings on bipartite graphs is due to Hopcroft and Karp [1973] runs in time $O\left(m\sqrt{n}\right)$ time; another which is faster depending upon the relationship between m and n computes max flows on graphs with unit capacities, due to Even and Tarjan [1975], runs in time $O\left(\min(m\sqrt{m}, mn^{2,3})\right)$. For the general graph matching problem, Micali and Vazirani [1980] designed an algorithm with runtime $O\left(m\sqrt{n}\right)$, and Mucha and Sankowski [2006] use fast matrix multiplication to find maximum matchings in $O(n^{\omega})$ time.

2 Definitions

We need a bunch of basic definitions before we begin.

Definition 1 (Simple Graphs)**.** *A graph $G = (V, E)$ is called* simple *if it has no repeated edges.*

Definition 2 (Bipartite graph)**.** *A graph $H = (L, R, E)$ is bipartite if for any $(u, v) = e \in E$ we have $u \in L, v \in R$.*

The next definitions are specific to the study of matchings.

Definition 3. *A vertex v is covered (open) with respect to a matching M if $(u, v) \in M$ for some $u \in V$ (no $(u, v) \in M$ for any u).*

Definition 4. *M is:*

1. *a* perfect matching *if $|M| = \frac{n}{2}$*
2. *a* maximum matching *if $|M| \geq |M'|$ for all matchings M' of G*
3. *a* maximal matching *if there is no $e \in E \setminus M$ for which $M \cup \{e\}$ is a matching.*

3 Matchings in Bipartite Graphs

3.1 Using Flow (and Ford-Fulkerson)

When G is bipartite, we can construct the flow network drawn in Section 3.1, with unit capacity on each edge. A max flow will have at most unit flow to (and from) any original vertex from G, so no vertex will be "overmatched"; then, integrality of flow implies there is always a max flow which is integral if the capacities are integral, so some max flow entirely uses the capacity of an edge or doesn't use it at all, for every edge. This is a matching, and Ford and Fulkerson [2009] can find it.

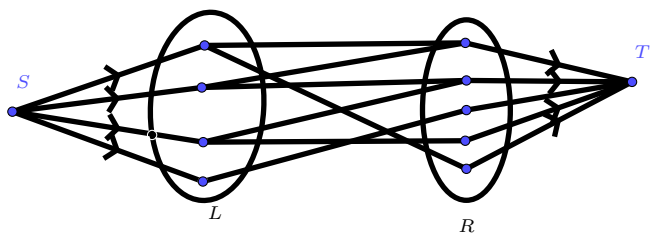

Figure 1: Flow network for a bipartite graph whose max flow corresponds to a maximum matching in G

3.2 König's algorithm

The algorithm we now present, König's algorithm[Konig, 1931], is a combinatorial one which introduces a number of ideas which will be useful for thinking about general (nonbipartite graphs). The first such ideas are the concepts of alternating and augmenting paths:

Definition 5 (Alternating and Augmenting Paths)**.** *A path P is M-alternating if its edges alternate between M and $E \setminus M$. P is M-augmenting if it is alternating and both endpoints of P are open in M.*

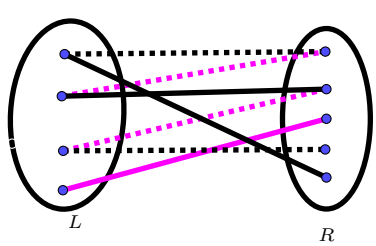

Figure 2: Pink edges form a matching on this graph. Black edges connote non-match edges in G. Dashed lines show one particular alternating (and augmenting) path for this matching.

It will also be useful to have compact notation for the symmetric difference between two sets.

Definition 6. *$S \Delta T = S \setminus T \cup T \setminus S$ is called the symmetric difference of S and T.*

jm pic alternating path

Notice that for a matching M and P an alternating path, $P \Delta M$ is a matching, such that $|P \Delta M| \geq |M|+1$ if P is augmenting.

Theorem 1 (Berge [1957])**.** *M is a maximum matching in G iff M has no augmenting path in G.*

Note that this theorem holds for general graphs, not simply bipartite graphs.

Proof. If there is an augmenting path P, the note that shows $P\Delta M$ is a matching of larger size, so M cannot be maximum.

In the other direction, suppose M is not maximum; we will construct an augmenting pat P for M. Consider M' a maximum matching; notice $|M'| > |M|$. Consider $S = M\Delta M'$; every vertex will have at most 2 incident edges in S So, it is a collection of paths and cycles. If any path is augmenting, we have found an augmenting path. If not, all the paths have an equal number of edges in M and M'.

In that case, consider the cycles. Since both M, M' are matchings, no vertex in any cycle has two edges from M incident to it (and similarly for M'). This means any vertex in the cycle, which must have degree two, has one edge incident to it from M and the other from M'. This implies the cycle must have even length! If not, it would not be possible for the edges to alternate between M and M' around the cycle. But this means that every cycle has an equal number of edges in M as in M', But if there is not an augmenting path and every cycle is of even length, $|M| = |M'|$, a contradiction.

So, there must exist an augmenting path for M. □

So, this suggests the following algorithmic template for finding maximum matchings: start with some matching, check if it has an augmenting path; if so, consider the symmetric difference between it and your current matching, and recurse. Since each augmenting path found increases the size of the current matching by at least 1, this is guaranteed to terminate, and by the previous theorem, we will always find an augmenting path if we aren't at a maximum matching. To complete this template into an actual algorithm, we need to specify how we find augmenting paths for a matching M efficiently.

For bipartite graphs, finding an augmenting path is rather straightforward. Begin by some open vertex v for M. Then, consider all outgoing edges from v not used in M. Call those level 0 edges and their other endpoints level 1 vertices. Then, consider all edges in M between level 1 vertices and vertices not in level 1 or v; call these vertices level 2 vertices. In general, consider any non-M edge between some vertex at level $2i$ and some "new" (not at level $0, \ldots, 2i$) vertex v', and add v' to level $2i+1$. From level $2i+1$, consider any M-edge between a vertex at level $2i+1$ and some new vertex v', add v' to level $2i+2$. This is written more formally in the following pseudocode:

Algorithm 1: LAYERED(G,M)

```
Let v be some open vertex w.r.t M;
Let S = S_0 = {v};
Let E' = ∅;
for i = 0; i < n ∧ S ≠ [n]; i++ do
    if i = 2j for some j ∈ ℕ then
        Add to E' all non-match edges (u_a, u_b) ∈ E \ M where u_a ∈ S_{2j} and u_b ∉ S;
    end
    else
        Add to E' all match edges (u_a, u_b) ∈ M where u_a ∈ S_i and u_b ∉ S;
    end
    Add those u_b to S_{i+1};
    Let S = S ∪ S_{i+1};
end
```

By construction, this graph will only contain alternating paths (all the edges are between adjacent levels, and all edges between one level and the next are the same "color", and at each level the "color" changes). If any vertex visited after level 0 is open, we have found an augmenting path. We just need to argue if there is an augmenting path for M this process will find one. We argue that if there is an alternating path between v and v' there will also be one in our layered graph. The edges which are removed from the original graph to form this layered graph are some non-match edges. If we were to add these back in, the graph would still

Algorithm 2: König(G,M)

Let $T = \text{LAYERED}(G, M)$;
if $\exists$ *an alternating path* P *from root* v *of* T *to open vertex* v' **then**
  return König$(P, P\Delta M)$;
end
else
  return M;
end

contain no odd cycles, since the graph is bipartite. So, any alternating path that uses these dropped edges could instead use the odd alternating edge path in the layered graph. This won't hold in general graphs, since adding these edges back can introduce odd cycles, for which one won't always have an alternating path starting and ending at the same places with incoming and outgoing non-match edges, respectively.

3.3 Edmond's Algorithm

As was informally alluded to in the previous section, the issue with running König's algorithm in general graphs will be that there are odd cycles in non-bipartite graphs; deleting non-match edges between vertices which are both in even or odd levels can change the alternating path connectivity in G. Fortunately, there's an elegant way to fix up this algorithm, presented by Edmonds [1987].

The idea will be to contract odd cycles and treat them as meta-vertices; the work comes in showing one does not introduce or lose augmenting paths by performing this action on a graph. More formally, a *flower* is a an an alternating path starting at an open vertex and ending with an odd-length cycle which is alternating for all but its two edges adjacent to the "root" of the cycle. The *stem* is the name for the alternating path, and a *blossom* is the name for the odd cycle. Here is an example:

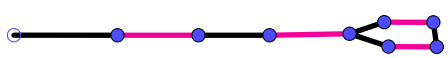

Figure 3: A flower: the endpoint is open, there's an alternating path(stem), followed by an odd cycle (blossom)

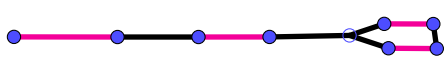

Figure 4: The flower "toggled", with the open vertex at the base of the blossom

Define the "toggling" of a flower to be the swapping of match edges for non-match edges along its stem, not changing the size of M but ensuring the open vertex is at the base of the cycle; this allows us to assume all flowers have length-zero stems. Define the contraction of a cycle C to be to replace V with $V \setminus C \cup \{v_C\}$, treating v_C as a new open vertex with all edges incident to any vertex outside of C.

Algorithm 3: Edmond(G,M)

$P = \text{AUGMENTING_PATH}(G, M)$;
If $P = \emptyset$ return M;
Else return $Edmond(G, P\Delta M)$;

Algorithm 4: AUGMENTING_PATH(G,M)

If $|V| = 1$ return $\emptyset$;
Let $T = \text{LAYERED}(G, M)$;
If T has an augmenting path P, return P;
Else if $T \cup E$ has a flower, toggle its stem and contract its blossom and recurse;
Else return $\emptyset$

Theorem 2. *Edmond's algorithm computes a maximum matching for any graph G in time $O(mn^2)$ time.*

To prove this algorithm is correct, we need to show that toggling a stem and contracting a blossom does not destroy or add any augmenting path in G, and that any time the algorithm says there is no augmenting path there actually isn't one. The runtime follows from the fact that building the layered structure takes $O(m)$ time, and it might be called $O(n)$ times before returning an augmenting path, and Edmonds might be called recursively $O(n)$ times (since each time the size of the matching increases by 1).

Proof. We begin by showing how to take any augmenting path through a contracted blossom and construct an augmenting path in the uncontracted graph, implying we don't add augmenting paths by these contractions. Every non-cycle edge adjacent to any vertex in the uncontracted blossom must be a non-matching edge (since the base is open as we toggled the stem and all other vertices have an adjacent match edge in the cycle). Moreover, the meta-vertex is open, so it must be an endpoint of the augmenting path in the contracted graph. The augmenting path must end at a different open vertex (which does not belong to the contracted blossom), and connect to v_C with a non-match edge in G. That non-match edge either connects to the base of the blossom (in which case we're done) or it connects to a vertex adjacent to a match edge (in which case, take that edge and follow the cycle in that direction to the base of the blossom). Either way, we construct an alternating path that ends at an open vertex and started at an open vertex, thus we've constructed an augmenting path for G.

We now argue that if there is an augmenting path P before contracting a blossom, there is still an augmenting path in the contracted graph. If there is an augmenting path P in G, it continues to exist after toggling a stem, so it suffices to think about augmenting paths in the toggled graph. If the augmenting path doesn't go through the contracted blossom, it's still a valid augmenting path in the contracted vertex. So, assume P goes through the contracted blossom. Since there is only one open vertex in the blossom, at least one of the endpoints of P is outside of the blossom. But then, the path starting at that open vertex and ending at the contracted blossom vertex (which is by construction an open vertex) is an augmenting path. Thus, an augmenting path still exists in the contracted graph.

Finally, notice if there is no simple augmenting path (one that does not intersect with a blossom) nor a blossom then there cannot be an augmenting path in G, thus when AUGMENTING_PATHreturns $\emptyset$ there is no augmenting path for M, and M is therefore maximum by Theorem 1. □

References

Claude Berge. Two theorems in graph theory. *Proceedings of the National Academy of Sciences*, 43(9): 842–844, 1957.

Jack Edmonds. *Paths, Trees, and Flowers*, pages 361–379. Birkhäuser Boston, Boston, MA, 1987. ISBN 978-0-8176-4842-8.

Shimon Even and R Endre Tarjan. Network flow and testing graph connectivity. *SIAM journal on computing*, 4(4):507–518, 1975.

LR Ford and DR Fulkerson. Maximal flow through a network. In *Classic papers in combinatorics*, pages 243–248. Springer, 2009.

John E Hopcroft and Richard M Karp. An n^5/2 algorithm for maximum matchings in bipartite graphs. *SIAM Journal on computing*, 2(4):225–231, 1973.

Dénes Konig. Graphok es matrixok (hungarian)[graphs and matrices]. *Matematikai és Fizikai Lapok*, 38: 116–119, 1931.

Silvio Micali and Vijay V Vazirani. An $o(\sqrt{v}\|e\|)$ algoithm for finding maximum matching in general graphs. In *Foundations of Computer Science, 1980., 21st Annual Symposium on*, pages 17–27. IEEE, 1980.

Marcin Mucha and Piotr Sankowski. Maximum matchings in planar graphs via gaussian elimination. *Algorithmica*, 45(1):3–20, 2006.